

DS_HW3_資訊三_嚴聲遠_111703009

程式碼運作邏輯設計：

利用腳本(.py)執行分別的演算法，腳本亦分別負責跑出折線圖。

詳細完整程式碼、成果請至：https://github.com/spaces-lalala/2024DS_HW

程式詳細說明：

1. BST：

```
// 節點結構
struct Node {
    int value;
    Node *left;
    Node *right;

    Node(int val) : value(val), left(nullptr), right(nullptr) {}
};

typedef Node *addr;
```

a. Create：

```
addr CreateBST(int num) { return new Node(num); }
```

函式負責創建 BST 中的單個節點。當傳入數值 num 時，它會利用 new 關鍵字創建一個包含該數值的新節點，並初始化其左右子節點為空指針。這個函式用於構造樹的根節點。

b. Insert：

```
addr InsertBST(int num, addr root) {
    if (root == nullptr) {
        return new Node(num);
    }
    if (num < root->value) {
        root->left = InsertBST(num, root->left);
    } else if (num > root->value) {
        root->right = InsertBST(num, root->right);
    }
    // 如果 num 已存在，不做任何操作
    return root;
}
```

函式實現了將新節點插入到 BST 的功能。它接收一個數值 `num` 和樹的根節點指標 `root`。如果根節點為空，說明樹尚未建立，函式直接創建一個新節點作為根。反則，根據 BST 的性質，遞歸地將數值插入到左子樹或右子樹中，具體取決於數值是小於還是大於當前節點的值。如果數值已存在，函式不進行任何操作。最終，它返回更新後的根節點，確保樹的結構完整。

c. Print :

```
void PrintBST(addr root) {
    if (root == nullptr) {
        cout << "Tree is empty." << endl;
        return;
    }
    int height = HeightBST(root);
    cout << "Height of the tree: " << height << endl;
    queue<addr> q;
    q.push(root);
    cout << "Tree values by level:" << endl;
    while (!q.empty()) {
        int levelSize = q.size();
        bool hasNextLevel = false; // 用於檢查下一層是否有真實節點
        for (int i = 0; i < levelSize; ++i) {
            addr current = q.front();
            q.pop();

            if (current) {
                cout << current->value << " ";
                q.push(current->left);
                q.push(current->right);

                // 如果子節點中有非空，標記為需要繼續打印
                if (current->left || current->right) {
                    hasNextLevel = true;
                }
            } else {
                cout << "null ";
                q.push(nullptr);
                q.push(nullptr);
            }
        }
        cout << endl;
        // 如果下一層全為空，停止處理
        if (!hasNextLevel)
            break;
    }
}
```

函式以 level 遍歷的方式打印 BST 的節點值。首先，它會檢查樹是否為空，若是，直接輸出提示。否則，函式會先計算樹的高度並輸出，然後使用一個 queue 逐層處理節點。在每一層中，函式會打印該層的節點值，若節點為空則輸出 "null"，並將左右子節點加入佇列。如果某層的所有節點均為空，打印會停止，從而避免多餘的輸出。

d. Height(root 為第一層，以下開始一層+1)：

```
int HeightBST(addr root) {
    if (root == nullptr)
        return 0;
    int leftHeight = HeightBST(root->left);
    int rightHeight = HeightBST(root->right);
    return max(leftHeight, rightHeight) + 1;
}
```

此函式用於計算樹的高度，即從根節點到最深葉子節點的最長路徑所包含的節點數。它採用遞歸方式，首先檢查節點是否為空，若是則返回高度 0。否則，分別計算左右子樹的高度，並取兩者的較大值加 1 作為結果。

2. AVL：

```
struct Node {
    int value;
    int height;
    Node *left;
    Node *right;

    Node(int val) : value(val), height(1), left(nullptr), right(nullptr) {}
};

// 計算節點高度
Tabnine | Edit | Test | Explain | Document | Ask
int getHeight(addr node) { return node == nullptr ? 0 : node->height; }

// 計算平衡因子
Tabnine | Edit | Test | Explain | Document | Ask
int getBalance(addr node) {
    return node == nullptr ? 0 : getHeight(node->left) - getHeight(node->right);
}

// 更新節點高度
Tabnine | Edit | Test | Explain | Document | Ask
void updateHeight(addr node) {
    if (node != nullptr) {
        node->height = max(getHeight(node->left), getHeight(node->right)) + 1;
    }
}
```

```
// 左旋
Tabnine | Edit | Test | Explain | Document | Ask
addr rotateLeft(addr x) {
    addr y = x->right;
    addr T2 = y->left;

    y->left = x;
    x->right = T2;

    updateHeight(x);
    updateHeight(y);

    return y;
}
```

```
// 右旋
Tabnine | Edit | Test | Explain | Document | Ask
addr rotateRight(addr y) {
    addr x = y->left;
    addr T2 = x->right;

    x->right = y;
    y->left = T2;

    updateHeight(y);
    updateHeight(x);

    return x;
}
```

a. Create :

```
addr CreateAVL(int num) { return new Node(num); }
```

這個函數用於創建一個新的 AVL 樹節點，並設置節點的初始值。當傳入數值 num 時，函數會呼叫節點構造函數，將 value 設置為 num，並將節點的高度設置為 1，左右子節點初始化為空指標 (nullptr)。該函數通常用於初始化樹的根節點，為 AVL 樹的建立提供基礎。

b. Insert :

```
addr InsertAVL(int num, addr root) {  
    if (root == nullptr) {  
        return new Node(num);  
    }  
  
    if (num < root->value) {  
        root->left = InsertAVL(num, root->left);  
    } else if (num > root->value) {  
        root->right = InsertAVL(num, root->right);  
    } else {  
        // 重複值，不插入  
        return root;  
    }  
  
    updateHeight(root);  
  
    int balance = getBalance(root);
```

```
    // LL Case  
    if (balance > 1 && num < root->left->value) {  
        return rotateRight(root);  
    }  
  
    // RR Case  
    if (balance < -1 && num > root->right->value) {  
        return rotateLeft(root);  
    }  
  
    // LR Case  
    if (balance > 1 && num > root->left->value) {  
        root->left = rotateLeft(root->left);  
        return rotateRight(root);  
    }  
  
    // RL Case  
    if (balance < -1 && num < root->right->value) {  
        root->right = rotateRight(root->right);  
        return rotateLeft(root);  
    }  
  
    return root;  
}
```

這個函數用於將新節點插入到 AVL 樹中，並確保樹在插入後仍然保持平衡。首先，它根據傳入的數值判斷新節點應插入到左子樹還是右子樹，若節點已存在則直接返回。插入後，更新當前節點的高度，並計算其平衡因子。如果發現不平衡（平衡因子的絕對值超過 1），則根據平衡因子的值和新插入節點的位置執行相應的旋轉操作（如 LL、RR、LR、RL 四種情況），從而保持 AVL 樹的特性。

c. Print :

```
void PrintAVL(addr root) {
    if (root == nullptr) {
        cout << "Tree is empty." << endl;
        return;
    }

    int height = HeightAVL(root);
    cout << "Height of the tree: " << height << endl;

    queue<addr> q;
    q.push(root);

    cout << "Tree values by level:" << endl;
    while (!q.empty()) {
        int levelSize = q.size();
        bool hasNextLevel = false; // 用於檢查下一層是否有真實節點
        for (int i = 0; i < levelSize; ++i) {
            addr current = q.front();
            q.pop();

            if (current) {
                cout << current->value << " ";
                q.push(current->left);
                q.push(current->right);

                // 如果子節點中有非空，標記為需要繼續打印
                if (current->left || current->right) {
                    hasNextLevel = true;
                }
            }
            else {
                cout << "null ";
                q.push(nullptr);
                q.push(nullptr);
            }
        }
        cout << endl;

        // 如果下一層全為空，停止處理
        if (!hasNextLevel)
            break;
    }
}
```

函式以 level 遍歷的方式打印 AVL 的節點值，並輔助檢查樹的高度和節點分佈狀況。它首先計算並輸出樹的高度，然後利用一個 Queue 逐層處理節點，按層次順序打印每個節點的值。如果某節點為空，輸出 "null"，並將其左右子節點作為空節點加入佇列。當所有節點的子樹均為空時，打印停止。

d. Height(root 為第一層，以下開始一層+1)：

```
int HeightAVL(addr root) { return getHeight(root); }
```

這個函數負責返回 AVL 樹的高度。高度的計算是基於節點中已存儲的

height 屬性，因此能夠高效地獲取結果。如果節點為空，則高度為 0；否則，直接返回節點的高度值。這種實現避免了每次都需要遞歸計算高度，能夠在插入和旋轉操作中快速進行平衡因子的計算，從而提高 AVL 樹的性能。

3. Treap :

```
// 節點結構
struct Node {
    int value;
    float priority;
    Node *left;
    Node *right;

    Node(int val, float pri) : value(val), priority(pri), left(nullptr), right(nullptr) {}
};
```

```
// 右旋
Tabnine | Edit | Test | Explain | Document | Ask
addr rotateRight(addr y) {
    addr x = y->left;
    addr T2 = x->right;

    x->right = y;
    y->left = T2;

    return x;
}
```

```
// 左旋
Tabnine | Edit | Test | Explain | Document | Ask
addr rotateLeft(addr x) {
    addr y = x->right;
    addr T2 = y->left;

    y->left = x;
    x->right = T2;

    return y;
}
```

a. Create :

```
addr CreateTreap(int num, float pri) { return new Node(num, pri); }
```

這個函數負責創建一個新的節點，接受兩個參數：節點的值 num 和對應的優先權 pri。函數通過初始化節點結構中的 value、priority、left 和 right 字段，生成一個新節點，並返回指向該節點的指標。這是構建 Treap 的基礎，為每個節點分配記憶體並設置初始值。

b. Insert :

```
// 2.InsertTreap
Tabnine | Edit | Test | Explain | Document | Ask
addr InsertTreap(int val, float pri, addr root) {
    if (root == nullptr) {
        return new Node(val, pri);
    }

    if (val < root->value) {
        root->left = InsertTreap(val, pri, root->left);
        if (root->left->priority < root->priority) {
            root = rotateRight(root);
        }
    } else if (val > root->value) {
        root->right = InsertTreap(val, pri, root->right);
        if (root->right->priority < root->priority) {
            root = rotateLeft(root);
        }
    }

    return root;
}
```

此函數實現將新節點插入到 Treap 的操作，並確保 Treap 的性質得以維護（根據值維持 BST 的性質，根據優先權維持 Min Heap 的性質）。首先，函數根據插入值 `val` 判斷應該遞迴插入到左子樹還是右子樹。插入完成後，會檢查子樹的優先權，若子樹的優先權小於當前節點的優先權，則使用右旋或左旋來調整樹的結構，確保 Min Heap 性質。

c. Print :

```
// 3.PrintTreap
tabnine | Edit | Test | Explain | Document | Ask
void PrintTreap(addr root) {
    if (root == nullptr) {
        cout << "Tree is empty." << endl;
        return;
    }

    cout << "Tree values by level:" << endl;
    queue<addr> q;
    q.push(root);

    while (!q.empty()) {
        int levelSize = q.size();
        bool hasNonNull = false; // 標記是否存在非空節點
        vector<string> levelOutput; // 暫存該層的輸出

        for (int i = 0; i < levelSize; ++i) {
            addr current = q.front();
            q.pop();

            if (current) {
                levelOutput.push_back(to_string(current->value) + "(" + to_string(current->priority) + ")");
                q.push(current->left);
                q.push(current->right);

                hasNonNull = true; // 該層有非空節點
            } else {
                levelOutput.push_back("null");
                q.push(nullptr); // 繼續放入 nullptr 保持完整結構
                q.push(nullptr);
            }
        }

        // 如果該層存在非空節點，才輸出
        if (hasNonNull) {
            for (const string& s : levelOutput) {
                cout << s << " ";
            }
            cout << endl;
        } else {
            break; // 當整層都是 null 時，停止
        }
    }
}
```

此函數以 Level 遍歷（Level Order Traversal）的方式列印 Treap 的節點，顯示每個節點的值和優先權，並使用 `null` 標註空節點。函數以佇列為基礎，逐層輸出每一層的節點資料，提供清晰的樹結構視覺化。如果樹為空，則直接輸出提示訊息。

d. Height(root 為第一層，以下開始一層+1)：

```
// 4.HeightTreap
int HeightTreap(addr root) {
    if (root == nullptr) {
        return 0;
    }
    return max(HeightTreap(root->left), HeightTreap(root->right)) + 1;
}
```

此函數計算 Treap 的高度，是一個典型的遞迴操作。函數會依次計算左子樹和右子樹的高度，並取二者的最大值，然後加 1 作為當前節點的高度。高度的計算有助於衡量 Treap 的平衡性和深度特性，從而分析操作的效率。

4. SkipList :

```
class Node {
public:
    int key;
    bool isRandom;
    int level;

    // Array to hold pointers to node of different level
    Node **forward;
    Node(int, int, bool);
};

Node::Node(int key, int level, bool isRandom) {
    this->key = key;
    this->level = level;
    this->isRandom = isRandom;

    // Allocate memory to forward
    forward = new Node *[level + 1];

    // Fill forward array with 0(NULL)
    memset(forward, 0, sizeof(Node *) * (level + 1));
};
```

```
// Class for Skip list
class SkipList {
    // Maximum level for this skip list
    int MAXLVL;

    // P is the fraction of the nodes with level
    // i pointers also having level i+1 pointers
    float P;

    // current level of skip list
    int level;

public:
    SkipList(int, float);
    int randomLevel();
    Node *createNode(int, int, bool);
    void InsertSkipList(int, bool useRandom = true, int rlevel = 0);
    void PrintSkipList();
    int HeightSkipList(Node *root);
    // pointer to header node
    Node *header;
};
```

```
SkipList::SkipList(int MAXLVL, float P) {
    this->MAXLVL = MAXLVL;
    this->P = P;
    level = 0;

    // create header node and initialize key to -1
    header = new Node(-1, MAXLVL, true);
};

// create random level for node
int SkipList::randomLevel() {
    float r = (float)rand() / RAND_MAX;
    int lvl = 0;
    while (r < P && lvl < MAXLVL) {
        lvl++;
        r = (float)rand() / RAND_MAX;
    }
    return lvl;
};
```


定義了SkipList的基本結構和核心機制，包括節點的設計、SkipList的初始化以及隨機層級的生成邏輯：

Node 類別:

節點是SkipList的基本組件。每個節點包含三個屬性：key用於排序，布林值 (isRandom) 用於記錄是否使用隨機層級，和節點的層級數 (level) 表示此節點可以連接的層級數。節點還有一個指標陣列 (forward)，用於連接SkipList中對應層級的下一個節點。建構子初始化這些屬性並分配記憶體給指標陣列。

SkipList 類別:

SkipList的核心類別，用於維護和操作節點。它包含以下屬性：

MAXLVL 是SkipList的最大層級，限制節點的最高層級。

P 是控制節點層級分佈的隨機概率參數，用於保持SkipList的平衡。

level 是SkipList當前的最高層級，根據插入操作動態調整。

header 是一個特殊的頭節點，提供進入SkipList的入口。

建構子負責初始化這些屬性，並創建頭節點，其鍵值為 -1，層級為 MAXLVL。

a. Create :

```
Node *SkipList::createNode(int key, int level, bool isRandom) {
    Node *n = new Node(key, level, isRandom);
    return n;
};
```

此函數負責創建一個新的節點，接受三個參數：節點的鍵值 key、節點的層級 level，以及是否使用隨機層級的標誌 isRandom。函數使用這些參數初始化節點的屬性，並為節點的 forward 指針陣列分配記憶體（大小為 level+1），以便建立指向其他節點的鏈接。最終返回新創建的節點指標。

b. Insert :

```
// 2.InsertSkipList
Tabnine | Edit | Test | Explain | Document | Ask
void SkipList::InsertSkipList(int key, bool useRandom, int rlevel) {
    if (useRandom) {
        rlevel = randomLevel();
    }

    Node *current = header;

    // create update array and initialize it
    Node *update[MAXLVL + 1];
    memset(update, 0, sizeof(Node *) * (MAXLVL + 1));

    for (int i = level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key < key)
            current = current->forward[i];
        update[i] = current;
    }

    current = current->forward[0];

    if (current == NULL || current->key != key) {
        if (rlevel > level) {
            for (int i = level + 1; i < rlevel + 1; i++)
                update[i] = header;
            level = rlevel;
        }

        Node *n = createNode(key, rlevel, useRandom);

        for (int i = 0; i <= rlevel; i++) {
            n->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = n;
        }
        // cout << "Successfully Inserted key " << key << " with level " << rlevel
        //      << "\n";
    }
};
```

此函數負責在 SkipList 中插入新節點，並根據需要決定節點的層級。

如果 useRandom 為 true，則使用 randomLevel 函數生成節點的隨機層級；否則使用指定的層級 rlevel。函數首先在每個層級尋找適合插入的位置，更新需要修改的鏈接。然後根據層級插入節點，並更新相關指針，確保 SkipList 結構的完整性。

c. Print :

```
// 3.PrintSkipList
Tabnine | Edit | Test | Explain | Document | Ask
void SkipList::PrintSkipList() {
    // cout << "\n*****Skip List*****" << "\n";
    for (int i = 0; i <= level; i++) {
        Node *node = header->forward[i];
        cout << "Level " << i << ": ";
        while (node != NULL) {
            cout << node->key << " ";
            node = node->forward[i];
        }
        cout << "\n";
    }
};
```

此函數用於列印 SkipList 的結構。函數會逐層輸出每一層的節點鏈接，從 header 開始沿著 forward 指針遍歷節點，並顯示每個節點的鍵值。這樣可以直觀地檢視 SkipList 中各層的分佈與連結。

d. Height(root 為第一層，以下開始一層+1) :

```
// 4.HeightSkipList
Tabnine | Edit | Test | Explain | Document | Ask
int SkipList::HeightSkipList(Node *root) {
    if (!root)
        return 0;
    int height = 0;
    for (int i = MAXLVL; i >= 0; i--) {
        if (root->forward[i] != NULL) {
            height = i + 1;
            break;
        }
    }
    return height;
}
```

此函數用於計算給定節點 root 所在的 SkipList 的高度。函數從最高層級開始向下檢查每一層，直到找到第一個非空的 forward 指針，並將該層級加一作為 SkipList 的高度。這用於評估 SkipList 的層級分佈，並對其平衡性進行分析。

5. 主函式(Main)：在所有的 main() 函式中，程序都通過命令列參數來接收測試的k以開始進行Insert。
6. 腳本：腳本的核心目的是透過執行 C++ 程式來分析不同演算法的表現。腳本使用 Python 來管理執行過程、處理結果，最後將數據視覺化跑出折線圖。

測試程式碼正確性(以下測試檢查後皆為正確)

1. BST：

測試code：

```
void testBST(void) {  
    cout << "Testing BST" << endl;  
  
    addr root = CreateBST(3);  
    root = InsertBST(2, root);  
    root = InsertBST(1, root);  
    root = InsertBST(5, root);  
    root = InsertBST(4, root);  
    PrintBST(root);  
    int height = HeightBST(root);  
    cout << "Height: " << height << endl;  
}
```

測試結果：

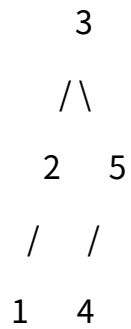
```
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> g++ .\BST.cpp  
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> ./a.exe  
Testing BST  
Height of the tree: 3  
Tree values by level:  
3  
2 5  
1 null 4 null  
Height: 3
```

由上到下分別為

Root(第一層)：3

第二層：2 5

第三層：1 空 4 空



樹高=3(root開始算1)

2. AVL :

測試code :

```
void testAVL(void) {
    cout << "Testing AVL" << endl;

    addr root = CreateAVL(1);
    PrintAVL(root);
    root = InsertAVL(2, root);
    PrintAVL(root);
    root = InsertAVL(3, root);
    PrintAVL(root);
    root = InsertAVL(4, root);
    PrintAVL(root);
    root = InsertAVL(5, root);
    PrintAVL(root);
    root = InsertAVL(6, root);
    PrintAVL(root);
    root = InsertAVL(7, root);
    PrintAVL(root);
    int height = HeightAVL(root);
    cout << "Height: " << height << endl;
}
```

測試結果 :

```

PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> g++ .\AVL.cpp
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> ./a.exe
Testing AVL
Height of the tree: 1
Tree values by level:
1
Height of the tree: 2
Tree values by level:
1
null 2
Height of the tree: 2
Tree values by level:
2
1 3
Height of the tree: 3
Tree values by level:
2
1 3
null null null 4
Height of the tree: 3
Tree values by level:
2
1 4
null null 3 5
Height of the tree: 3
Tree values by level:
4
2 5
1 3 null 6
Height of the tree: 3
Tree values by level:
4
2 6
1 3 5 7
Height: 3

```

由上到下分別為

Root(第一層) : 4

第二層 : 2 6

第三層 : 1 3 5 7

4

/\

2 6

/\ /\

1 3 5 7

樹高=3(root開始算1)

3. Treap :

測試code :

```
void testTreap() {  
    addr root = nullptr;  
  
    root = InsertTreap(3, 0.9, root);  
    root = InsertTreap(2, 0.5, root);  
    root = InsertTreap(1, 0.3, root);  
    root = InsertTreap(5, 0.2, root);  
    root = InsertTreap(4, 0.1, root);  
  
    // 列印 Treap  
    PrintTreap(root);  
  
    // 計算高度  
    int height = HeightTreap(root);  
    cout << "Height: " << height << endl;  
}
```

測試結果 :

```
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> g++ .\Treap.cpp  
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> ./a.exe  
Tree values by level:  
4(0.100000)  
1(0.300000) 5(0.200000)  
null 2(0.500000) null null  
null null null 3(0.900000) null null null null  
Height: 4
```

由上到下(括號內為pri)分別為

Root(第一層) : 4(0.1)

第二層 : 1(0.3) 5(0.2)

第三層 : 空 2(0.5) 空 空

第四層 : 空 空 空 3(0.9) 後面都空

```
      4(0.1)  
     /   \  
  1(0.3) 5(0.2)  
   \  
    2(0.5)  
   \  
    3(0.9)
```

樹高=4(root開始算1)

4. SkipList :

測試code :

```
void testSkipList() {
    SkipList lst(100, 0.5);

    // 插入指定層級的節點
    lst.InsertSkipList(1, false, 2); // HHT
    lst.InsertSkipList(2, false, 0); // T
    lst.InsertSkipList(3, false, 1); // HT
    lst.InsertSkipList(4, false, 3); // HHHT
    lst.InsertSkipList(5, false, 0); // T
    lst.InsertSkipList(6, false, 0); // T
    lst.InsertSkipList(7, false, 2); // HHT
    lst.PrintSkipList();

    // Test HeightSkipList
    cout << "Height of SkipList: " << lst.HeightSkipList(lst.header) << endl;
};
```

測試結果 :

```
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> g++ .\SkipList.cpp
PS C:\Users\USER\vscode workspace\NCCUCP\data_structure\2024DS_HW\hw3> ./a.exe
Level 0: 1 2 3 4 5 6 7
Level 1: 1 3 4 7
Level 2: 1 4 7
Level 3: 4
Height of SkipList: 4
```

由上到下分別為

第四層:								4
第三層:		1		4				7
第二層:		1	3	4				7
第一層:	1	2	3	4	5	6	7	

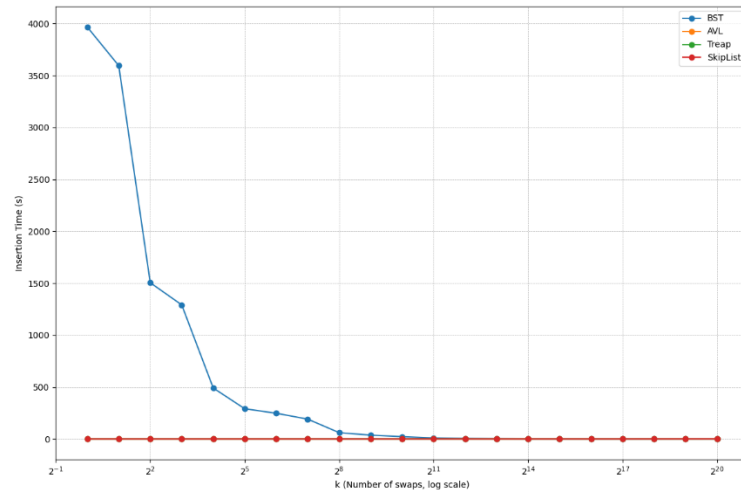
樹高=4

5. 實驗圖分析：

詳細折線圖與數據同前文連結：https://github.com/spaces-alala/2024DS_HW

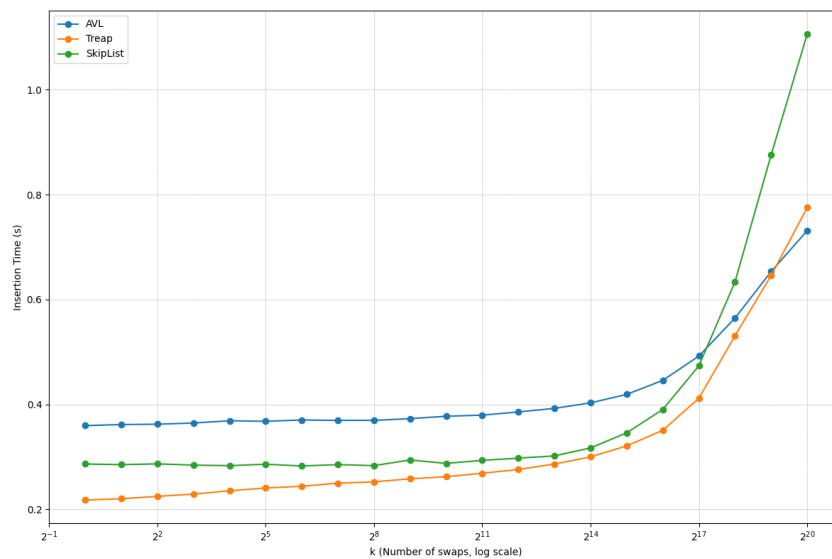
1. 第一張折線圖：

折線圖：



觀察 csv 後發現，由於 BST 數值過大，壓縮了其他資料結構的顯示，所以我做了一個去除 BST 的版本。

折線圖(without BST)：



分析：

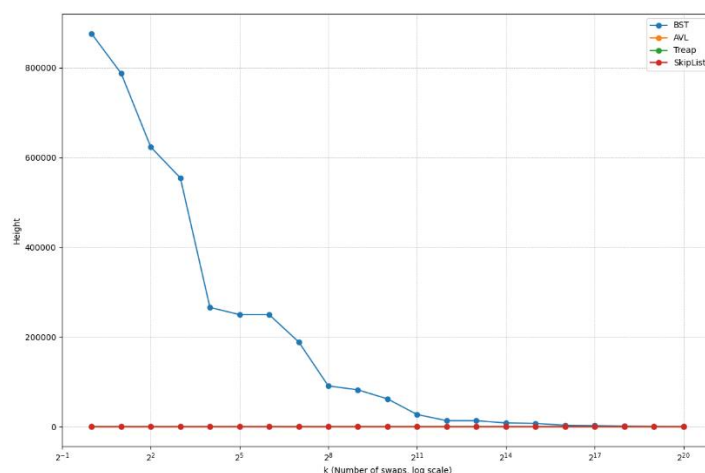
首先分析 **BST**，BST 在接近排序($k=0$)時的所需時間最長，而隨著 k 增加所需

時間也隨之大幅下降，BST 在初始排序數據插入時退化為鏈結結構，因此新增時間很長，而隨著隨機化增加，BST 能保持較好的平衡，插入時間短很多。

BST 不如 AVL 及 Treap 有相關平衡機制，也不如 SkipList 維持在 $O(\log n)$ ，故所需時間多上很多。而 **AVL** 由於有透過旋轉進行平衡機制。插入時間相對穩定，雖然相差不大，但在 k 較小時(幾乎排序好)，可能會導致 rotation 需要較多故時間可能需要多一點點，由於 AVL 需要相對較多的頻繁旋轉，故相對 Treap、SkipList 需要較多時間。而 **Treap** 結合了 BST 的結構和最小堆 (Min-Heap) 的隨機性。即使資料排序，隨機的優先級會破壞退化情況，保持較好的插入時間。隨著隨機化增加，插入時間穩定在 $O(\log n)$ 附近。雖然 rotation 會相對較 AVL 少，但依舊需要平衡調整故時間高於 SkipList。最後 **SkipList** 的插入時間幾乎不受數據的排列方式影響。由於其結構是基於機率的，無論數據是否排序，插入時間始終保持在 $O(\log n)$ 範圍內，且不會隨 k 的變化而大幅波動。SkipList 的調整完全依賴隨機過程，時間複雜度穩定為 $O(\log n)$ ，且沒有旋轉或調整堆性質的額外開銷。故時間花費最少。

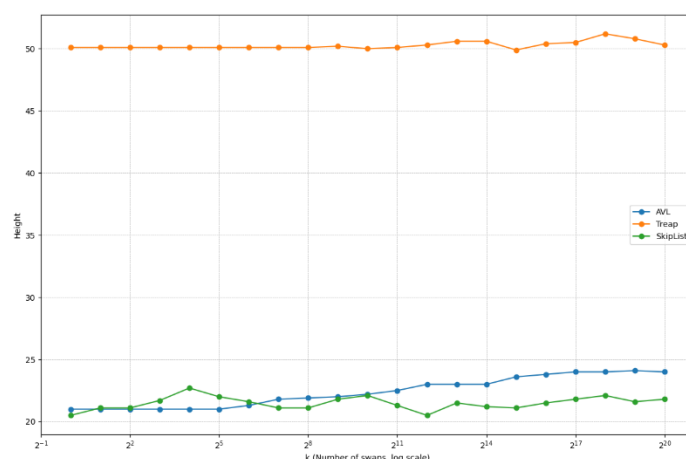
關於為甚麼都會在 k 越來越大時間也越來越多，當 k 很小時 (接近排序狀態)：新插入的節點大多會直接加在右子樹，需要的旋轉操作較少 CPU 分支預測較準確，因為插入模式很規律，當 k 變大 (越來越隨機) 時：需要更多的操作來維持平衡，插入路徑變得不規律，增加了 CPU 分支預測失誤(cache miss)，記憶體存取模式變得不連續，降低了 CPU 快取效率，但這部分還尚未測試釐清(放至問題討論)。

2. 第二張折線圖：



觀察 csv 後發現，由於 BST 數值過大，壓縮了其他資料結構的顯示，所以我做了一個去除 BST 的版本。

折線圖(without BST)：



分析：

對於 **BST** 而言，如同執行時間一樣，結構高度隨著 k 值增加而下降，隨著 k 增加，數據更隨機化，BST 接近理想的平衡狀態，高度開始接近 $O(\log n)$ 。而對於 **Treap**，由於我使用的是隨機優先權 (uniform_real_distribution)，插入的數列順序（升序或隨機）對最終 Treap 的形狀影響較小，主要由隨機優先權決定。故高度不論 k 值大小相對穩定。但相對 AVL 和 Skiplist 都有確定性或概率性的結構維護機制：(AVL 嚴格保持平衡因子，任何插入或刪除後會進行

必要的旋轉，平衡度極高。Skiplist 的多層結構是基於概率)，Treap 僅依賴局部旋轉操作（基於隨機優先權）來維護堆特性。這種局部維護機制並不保證整棵樹的平衡，導致高度偏高。而 AVL 結構高度幾乎不變，隨 k 略微波動。數據的隨機化對 AVL 的平衡影響很小，導致高度穩定。Skiplist 的高度基於隨機層次結構生成，無論數據是否排序，故與 AVL 一樣高度都保持在 $O(\log n)$ ，並且不會隨 k 增大而有顯著變化。

問題：

1. 圖一觀察到關於隨著 k 值越大，AVL、Treap、Skiplist有顯著的上升，由於若不考慮快取，此三種資料結構本身的時間並不如BST所需的時間長，故存取快取(cache miss)的影響可能因此佔比較重，故有向上趨勢。但以上僅是推論，不確定是否真的是cache的影響很重。關於Skiplist在 k 值接近 2^{20} 時為何開始遠高出Treap以及AVL也尚須測試。
2. 由於電腦本身的設定，在測試BST時須特別設定遞迴深度限制，不確定為了BST開啟的設定是否會造成過多開銷而影響時間的測量結果。